

D2 Baseline: Numerical Method

The calculator computes the principal real value of $\arccos(x)$ for $-1 \leq x \leq 1$, with results in $[0, \pi]$. It uses range reduction, a Taylor-series recurrence for arcsin, and Newton’s method for square roots without production use of `math`.

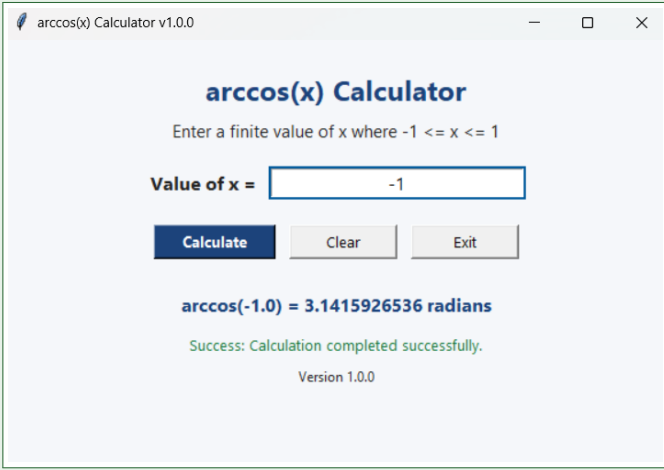
$$\arccos(x) = \begin{cases} 2 \arcsin\left(\sqrt{(1-x)/2}\right), & x \geq 0, \\ \pi - \arccos(-x), & x < 0. \end{cases}$$

Exact endpoint handling returns $\arccos(1) = 0$ and $\arccos(-1) = \pi$. The tested error target is $\leq 10^{-6}$ radians.

D3 Problem 7: Software Quality and Accessibility

Problem 7 improves the D2 calculator through PEP 8, Flake8, Pylint, `pdb`, Semantic Versioning, UIDP-guided interface design, and accessibility-oriented improvements. The production implementation remains from scratch: it does not import `math` and does not use library inverse-trigonometric or square-root functions.

GUI Demonstration



Exact endpoint calculation: $\arccos(-1) = \pi = 3.1415926536$ radians.

Implementation and Versioning

- From-scratch implementation.** The application manually implements absolute value, square root, arcsine, and arccosine. Tkinter is used only for the interface; `math` is not imported by `problem7.py`.
- Validation and exceptions**
 - `InputValidationError`: NaN, infinity, or invalid domain
 - `ConvergenceError`: configured iteration limit reached
- Semantic Versioning**

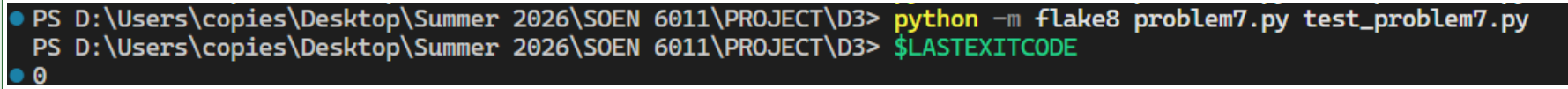
v1.0.0 (D2) $\longrightarrow$ v1.1.0 (D3)

Why minor? D3 adds backward-compatible quality and usability features: automated tests, static analysis, debugger evidence, keyboard support, visible focus, and clearer status feedback. The public calculator behavior and valid input domain are unchanged. The identifier **v1.1.0** is consistent in the code, GUI title, README, and Git tag.

PEP 8 and Flake8

Applied practices include four-space indentation, descriptive names, docstrings, grouped constants, wrapped expressions, and separation of numerical and GUI responsibilities.

`python -m flake8 problem7.py test_problem7.py`
Analyzed files: `problem7.py` and `test_problem7.py`.
Final result: *0 violations reported.*

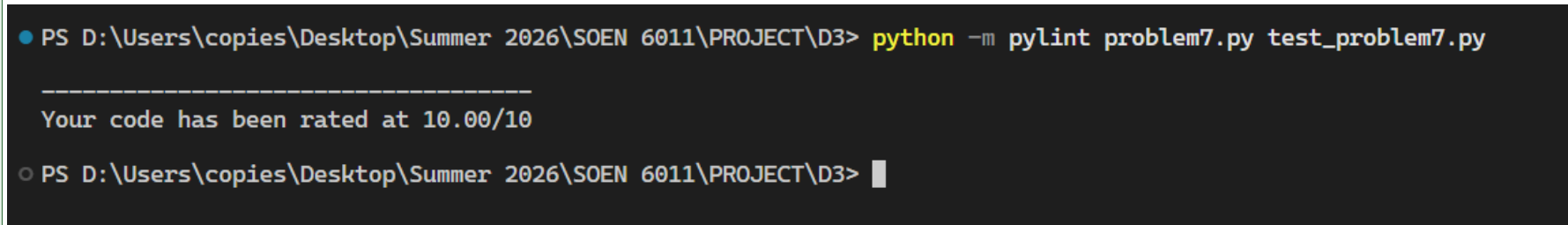


Flake8 execution evidence for the final source and test files.

Static Analysis with Pylint

Pylint inspected the final source and test files for errors, naming, documentation, unused code, exception handling, and maintainability.

`python -m pylint problem7.py test_problem7.py`
Analyzed files: `problem7.py` and `test_problem7.py`.
Final score: 10.00/10.
Remaining messages: None.

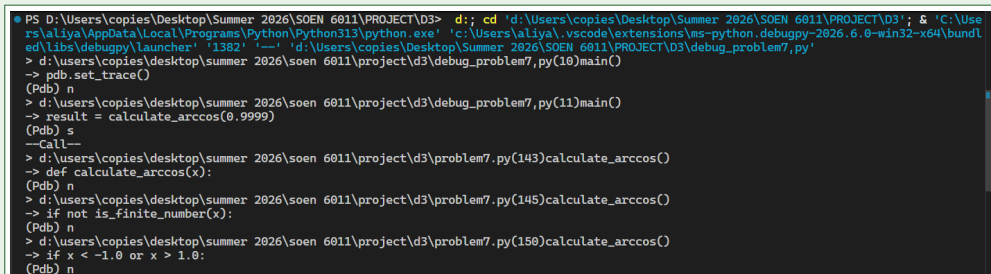


Pylint report for the final source and test files.

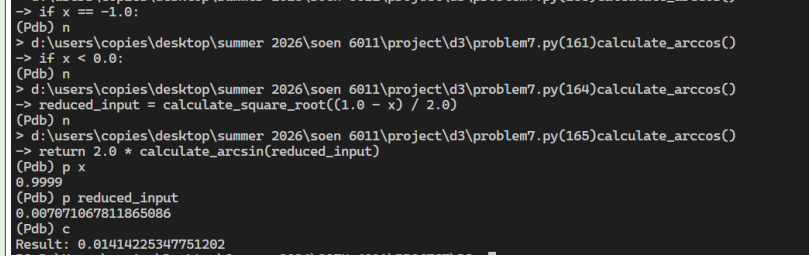
Debugging with pdb

A breakpoint was placed in `calculate_arccos()` immediately before range reduction for $x = 0.9999$. The session used `n`, `s`, `p`, and `c` to inspect `x`, `reduced_input`, `term`, `coefficient`, and `result`.
Confirmed: range reduction produces a small arcsine argument near the endpoint; Taylor terms decrease to the configured tolerance; and the final value remains in the principal range $[0, \pi]$. The investigation found no numerical defect.

Command	Purpose
n	Execute the next line
s	Step into a function
p variable	Inspect a variable
c	Continue execution



Breakpoint, validation, and range-reduction inspection.



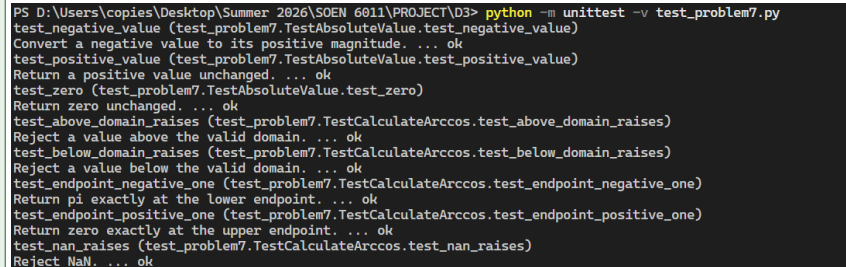
Taylor recurrence variables and final-result inspection.

D3 Problem 8: Unit Testing

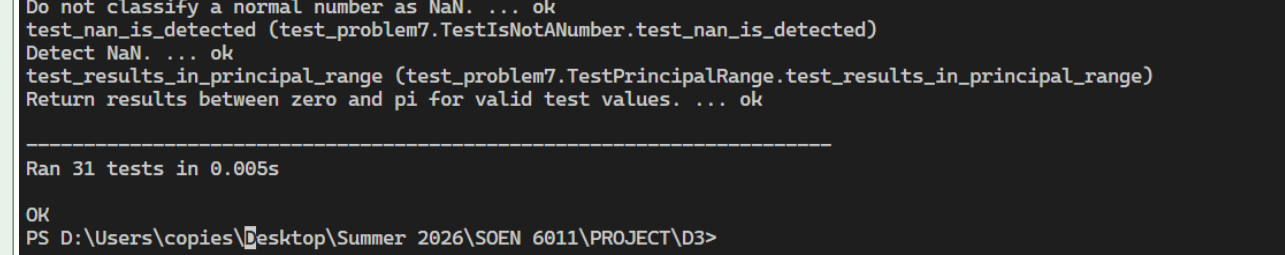
The automated suite uses Python’s built-in `unittest` framework (PyUnit) and imports the D3 production module `problem7.py`.
Coverage: helper functions, Newton square root, Taylor arc-sine, exact endpoints, normal inputs, near-endpoint inputs, invalid-domain values, NaN, infinity, principal range, numerical accuracy, and exception paths.

`python -m unittest -v test_problem7.py`

Result: 31 tests passed. Reference values from `math.acos()` and `math.asin()` are test oracles only; the production module `problem7.py` does not import `math`.



PyUnit evidence: first group of test cases.



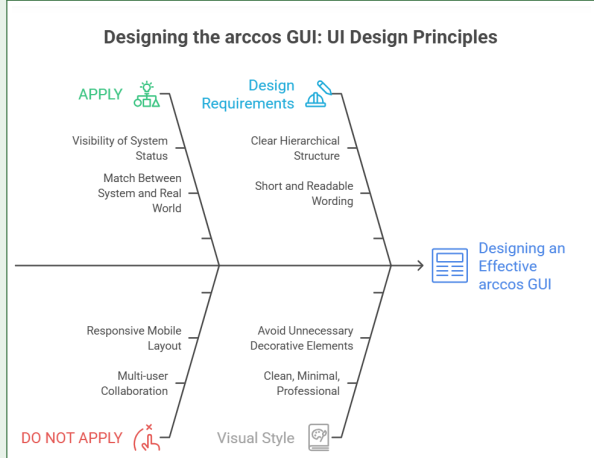
PyUnit evidence: final tests and OK result.

Selected Test Cases

ID	Input	Expected outcome
TC01	1.0	Exact result 0.0
TC02	-1.0	Exact result π
TC03	0.0	Approximately $\pi/2$
TC04	0.5 and -0.5	Approximately $\pi/3$ and $2\pi/3$
TC05	± 0.9999	Accurate near-endpoint result
TC06	1.1, -1.1	<code>InputValidationError</code>
TC07	NaN, $\pm\infty$	<code>InputValidationError</code>
TC08	Valid test set	Error $\leq 10^{-6}$ radians

UIDP and Accessibility

- Directly applied**
 - User Profiling:** supports users entering one real value x and expecting an angle in radians.
 - Humility:** clear guidance and recoverable errors avoid blaming the user.
 - Feature Exposure:** labelled input plus visible Calculate, Clear, and Exit controls.
 - State Visualization:** separate result and status areas show ready, success, and error states.
 - Coherence:** consistent labels, colours, shortcuts, and error wording.
 - Safety:** validation prevents invalid calculations while preserving input for correction.
- Limited applicability: Metaphor** is limited because a scientific calculator is already a familiar direct-manipulation model.
- Accessibility aim:** readable text, high contrast, visible focus, keyboard navigation, shortcuts, and textual feedback not based only on colour. The GUI *aims to be accessible*; it was not formally evaluated with users or assistive technology.



UIDP applicability decision mind map.

GAI and CASTROFF Evidence

AI tool: Perplexity AI.
Problem 7: Code Quality and Accessibility
Prompt: “Review my from-scratch Python Tkinter $\arccos(x)$ calculator for PEP 8, maintainability, UIDP, and accessibility. Do not recommend prohibited `math` functions.”
CASTROFF: *Constraints*—from-scratch code, no production `math`; *Audience*—SOEN 6011 evaluator; *Structure*—prioritized review; *Tone*—technical; *Role*—quality reviewer; *Output*—actionable recommendations; *Focus*—PEP 8, UIDP, and accessibility; *Function*—improve D3 quality.

Outcome: Accepted clearer feedback, visible focus, keyboard support, and documentation. Rejected suggestions violating the from-scratch constraint. Recommendations were checked against the code and compiled GUI.

Problem 8: Boundary and Exception Testing
Prompt: “Review my PyUnit tests and suggest boundary, invalid-input, exception, convergence, and accuracy tests. Use `math.acos()` and `math.asin()` only as test oracles.”

CASTROFF: *Constraints*—existing API, no production `math`; *Audience*—SOEN 6011 evaluator; *Structure*—boundary and exception tests; *Tone*—technical; *Role*—test reviewer; *Output*—executable PyUnit tests; *Focus*—boundaries, exceptions, accuracy, and range; *Function*—validate the implementation.

Outcome: Tests were reviewed against the implementation and executed locally: **31 tests passed**. The `math` functions were used only as test oracles, not production dependencies.

Conclusion

The D3 release combines a range-reduced Taylor series and Newton square root with validation, exception recovery, and a Tkinter GUI. Flake8, Pylint, `pdb`, Semantic Versioning, UIDP-guided design, and automated PyUnit tests provide verifiable quality evidence.

Result: a numerically reliable, maintainable, testable, and accessibility-oriented $\arccos(x)$ calculator.

References and Repository

- Concordia University, *SOEN 6011 Project Description*, Summer 2026.
 - Python Software Foundation, *PEP 8 – Style Guide for Python Code*.
 - PyCQA, *Flake8 Documentation*.
 - Pylint Contributors, *Pylint Documentation*.
 - Python Software Foundation, *pdb – The Python Debugger*.
 - Python Software Foundation, *unittest – Unit Testing Framework*.
 - Semantic Versioning, *Semantic Versioning 2.0.0*.
 - W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*.
 - SOEN 6011 Course Material, *CASTROFF Prompt Framework*.
- Repository:**
github.com/md-aliyawar-khan/SOEN-6011-D3-arccos-calculator